Vokabeltrainer als Webanwendung mit Vue.js

Andreleene Prado und Ronja Rüther
Fachhochschule Südwestfalen

Schriftliche Ausarbeitung im Modul
„Frontend-Frameworks für Webanwendungen“

14. März 2026

Inhaltsverzeichnis

1. Einleitung (Ronja Rüther) 3

2. Verwendete Bibliotheken und Werkzeuge (Andreleene Prado) 3

3. Aufbau der Webanwendung (Ronja Rüther) 4
3.1. Authentifizierung und Nutzerzugang (Ronja Rüther) 4
3.2. Dashboard (Ronja Rüther) 5
3.3. Vokabelverwaltung (Andreleene Prado) 5
3.4. Interaktive Lernmodule (Andreleene Prado) 5
3.4.1. Wortbasierte Lernmodi (Andreleene Prado) 6
3.4.2. Satzbasierte Lernmodi (Ronja Rüther) 6
3.4.3. Karteikarten (Andreleene Prado) 6
3.5. Chatbot (Andreleene Prado) 6
3.6. Profil (Ronja Rüther) 7
3.7. Einstellungen (Ronja Rüther) 7

4. Architektur und Implementierung (Ronja Rüther) 8
4.1. Systemarchitektur (Ronja Rüther) 8
4.2. Implementierung der Benutzerverwaltung (Ronja Rüther) 9
4.2.1. Authentifizierung (Ronja Rüther) 9
4.2.2. Profil-Logik (Ronja Rüther) 9
4.2.3. Benutzeroberfläche zur Kontoverwaltung (Ronja Rüther) 10
4.3. Implementierung der Vokabelverwaltung (Andreleene Prado) 10
4.4. Implementierung interaktiver Lernmodule (Andreleene Prado) 11
4.4.1. Dynamische Komponentenstruktur der Lernmodule (Andreleene Prado) . 11
4.4.2. Implementierung der Karteikarten (Andreleene Prado) 12
4.4.3. Implementierung der Spiellogik (Andreleene Prado) 12
4.4.4. Wortbasierte Lernmodi (Andreleene Prado) 14
4.4.5. Satzbasierte Lernmodi (Ronja Rüther) 15
4.5. Implementierung des Chatbots (Andreleene Prado) 16
4.6. Visualisierung der Lernstatistik (Ronja Rüther) 17
4.7. Implementierung des Abzeichensystems (Andreleene Prado) 18

5. Aufgetretene Probleme 19
5.1. E-Mail-Änderung mit Firebase (Ronja Rüther) 19
5.2. Globales Laden von Lernsets (Ronja Rüther) 19
5.3. Aktualisierung und Berechnung der Progressbar (Andreleene Prado) 20
5.4. Anzeige des Level-Up-Overlays (Ronja Rüther) 20

6. Ergebnisse und Fazit (Andreleene Prado) 21

7. Anhang (Ronja Rüther) 21
7.1. Verwendete grafische Assets 21

1. Einleitung (Ronja Rüther)

Digitale Sprachlernanwendungen gewinnen in den letzten Jahren stetig an Bedeutung zum selbstständigen Erlernen einer Fremdsprache. Da die meisten Personen heutzutage ein Smartphone verfügbar haben, lassen sich Vokabeln regelmäßig wiederholen und der Wortschatz festigen, da Wartezeiten so überall zum Lernen genutzt werden können. Die bekannte Anwendung *Duolingo* zeigt bereits, wie das digitale Erlernen einer neuen Sprache funktionieren kann. Um das Lernen einfacher und motivierender zu gestalten, werden Gamification-Elemente, wie z.B. das Zusammensetzen eines Satzes aus Wort-Bausteinen, verwendet. [1] Dennoch stößt diese Plattform an eine Grenze, denn die Lerninhalte sind statisch vorgegeben und der Nutzer kann keine eigenen Vokabeln anlegen, die er speziell für die Schule, den Beruf oder ein neues Hobby benötigt.

Ziel dieses Projekts ist daher die Entwicklung einer Webanwendung namens „Lingotrain“, die das Prinzip der interaktiven Sprachübungen mit der eigenen Verwaltung der Lerninhalte kombiniert. Statt vorgefertigter Lektionen ermöglicht diese Anwendung die Erstellung individueller Lernsets mit Vokabeln. Der Fokus liegt dabei auf den Zielsprachen Englisch, Französisch und Spanisch mit Deutsch als Ausgangssprache. Durch dynamisch generierte Übungen und Gamification-Aspekte können die Nutzer ihre eigenen Vokabeln im eigenen Tempo lernen. Damit die Vokabeln, insbesondere einzelne Wörter, auch kontextbezogen und im Hinblick auf eine Unterhaltung gelernt werden, verfügt die Webanwendung auch über einen Chatbot, der passende Sätze zu den einzelnen Wörtern generiert und dem Nutzer passendes Feedback geben kann. Darüber hinaus ist auch ein Konversationsmodus geplant, in dem der Nutzer mit dem Chatbot eine Unterhaltung zu einer bestimmten Vokabel aus seinen Lernsets führen kann.

Die technische Umsetzung von „Lingotrain“ stellt dabei hohe Anforderungen an die Architektur des verwendeten Frontend-Frameworks. Da neben statischen Vokabellisten auch reaktive Übungsmodule sowie asynchrone Chat-Schnittstellen nötig sind, wird eine modulare und komponentenbasierte Struktur benötigt. Im Rahmen dieser Ausarbeitung wird daher dokumentiert, wie die Anforderungen mithilfe eines modernen und komponentenorientierten Frameworks umgesetzt wurden, damit der Anwender die Webanwendung performant nutzen kann.

2. Verwendete Bibliotheken und Werkzeuge (Andreleene Prado)

Für die Entwicklung des Frontends der Web-Applikation wurde das Framework **Vue.js** verwendet. Diese Framework bietet eine komponentenbasierte Architektur, die eine strukturierte und wartbare Entwicklung ermöglicht. Zudem erleichtert das Framework durch seine reaktive Datenbindung die Implementierung dynamischer Benutzeroberflächen. [2]

Für die Navigation zwischen den einzelnen Seiten der Anwendung wurde der **Vue Router** eingesetzt. Dieser ermöglicht die Verwaltung verschiedener Routen innerhalb einer Single-Page-Application und sorgt dafür, dass Benutzer nahtlos zwischen den unterschiedlichen Ansichten der Anwendung wechseln können. [3]

Für bestimmte Benutzeroberflächenkomponenten wie Buttons, Dialoge und Checkboxen wurde die UI-Bibiliothek **vue-shadcn** eingesetzt. Diese stellt vorgefertige, moderne und anpassbare Komponenten zur Verfügung, wodurch eine konsistente Gestaltung der Benutzeroberfläche gewährleistet werden kann. Gleichzeitig reduziert die Verwendung solcher Komponenten den Implementierungsaufwand.

Für das Design und die Gestaltung der Benutzeroberfläche wurde **Tailwind CSS** verwendet. Tailwind ist eine Utility-First-CSS-Framework, das eine schnelle und flexible Gestaltung von Benutzeroberflächen ermöglicht. Durch vordefinierte Klassen können Layout, Abstände, Farben und Typografie effizient definiert werden, ohne umfangreiche eigene CSS-Strukturen erstellen zu müssen. [4]

Für das Zustandsmanagement der Anwendung wurden **Pinia Stores** verwendet. Pinia ist die offizielle State-Management-Lösung für Vue.js und ermöglicht eine zentrale Verwaltung des Anwendungszustands. Dadurch können Daten effizient zwischen verschiedenen Komponenten geteilt und verwaltet werden, was insbesondere bei der entwickelten Sprachlern-Anwedung die Struktur und Wartbarkeit des Codes verbessert. [5]

Als Datenbank wurde **Google Firestore** verwendet. Firestore ist eine cloudbasierte NOSQL-Datenbank, die eine einfache Speicherung und Synchronisierung von Daten in Echtzeit ermöglicht. Für den Zugriff auf die Datenbank wurden Methoden aus dem Firestore-SDK importiert und innerhalb von Repository-Klassen verwendet. Diese Repository-Struktur kapselt die Datenbankzugriffe und sorgt für eine klare Trennung zwischen Datenzugriff und Anwendungslogik.

Das Backend der Anwendung wurde mit **FastAPI** umgesetzt. FastAPI ist ein modernes Web-Framework für Python, das sich besonders für die Entwicklung performanter und leichtgewichtiger APIs eignet. Über diese Schnittstelle wird unter anderem die Chatbot-Funktionalität bereitgestellt, die im Frontend der Anwendung integriert ist. Dadurch kann due Chatbot-Logik serverseitig verarbeitet und die generierten Antworten anschließend im Benutzerinterface angezeigt werden. [6]

Zusätzlich wurden während der Implementierung unterstützend KI-basierte Werkzeuge wie ChatGPT und Copilot verwendet. Die KI-Tools dienten somit primär als Hilfsmittel im Entwicklungsprozess, beispielsweise bei kleineren Implementierungsfragen oder bei der Fehlersuche.

3. Aufbau der Webanwendung (Ronja Rüther)

Die Struktur der Webanwendung folgt einem modularen Aufbau und bietet eine übersichtliche Gruppierung der Kernfunktionen. Da *Lingotrain* als *Single-Page-Application* (SPA) umgesetzt wurde, erfolgt der Übergang zwischen den einzelnen Modulen ohne spürbare Neulade-Vorgänge. Die Benutzeroberfläche gliedert sich dabei in die folgenden fünf Hauptbereiche: *Home*, *Meine Vokabeln*, *Chatbot*, *Profil* und *Einstellungen*.

3.1. Authentifizierung und Nutzerzugang (Ronja Rüther)

Der Zugang zur Webanwendung wird durch einen Authentifizierungsprozess geschützt, der die Registrierung und Anmeldung mittels E-Mail-Adresse und Passwort umfasst. Beim initialen Aufruf der Anwendung bildet die Login-Seite den zentralen Einstiegspunkt, von der aus auch zur Registrierung weitergeleitet werden kann, sollte der Nutzer noch kein Konto besitzen.

Nach erfolgreicher Authentifizierung erfolgt in beiden Fällen eine automatische Weiterleitung auf das Dashboard (Home). Beim Login hat der Nutzer außerdem die Wahl, ob er für zukünftige Sitzungen angemeldet bleiben möchte oder nicht. Dies soll die Nutzerfreundlichkeit erhöhen, da ein vermehrter Anmeldeprozess häufig als störend empfunden wird. Durch die Speicherung der

Sitzungsdaten wird bei zukünftigen Aufrufen der Anwendung der Anmeldeprozess übersprungen und direkt zum Dashboard weitergeleitet.

Der Authentifizierungsprozess ist hier besonders wichtig, damit die individuell angelegten Lernsets und Vokabeln sowie die erreichten Fortschritte einem Benutzerkonto zugeordnet werden können. Durch diese eindeutige Identifizierung ist es möglich, die personalisierten Daten aus der Datenbank zu laden und dem Benutzer auf verschiedenen Endgeräten zur Verfügung stellen zu können.

3.2. Dashboard (Ronja Rüther)

Das Dashboard (Home) bildet den Einstiegspunkt für angemeldete Benutzer und dient zur Motivation sowie dem schnellen Einstieg in die Lerneinheiten. Direkt nach dem Login wird der Nutzer hier namentlich begrüßt und erhält eine Übersicht über seinen aktuellen Lernfortschritt. Hierzu zählen:

- **Gamification-Elemente:** Durch die Anzeige von Punkten, dem Level und der „Lernserie“ (Streak) soll der Nutzer zum regelmäßigen Lernen animiert werden.
- **Sprachauswahl:** Über ein Auswahlmenü kann der Nutzer zwischen den Sprachen, die er seinem Konto hinzugefügt hat, die aktuelle Lernsprache auswählen.
- **Quick-Access (Shortcuts):** Das Dashboard bietet einen Zugriff auf maximal drei angelegte Lernsets. Durch diese Shortcuts kann der Nutzer eine Lernrunde schneller starten und muss nicht den Umweg über den Tab *Meine Vokabeln* nehmen.

3.3. Vokabelverwaltung (Andreleene Prado)

Der Bereich *Meine Vokabeln* bietet eine Übersicht über alle vom Benutzer angelegten Lernsets bzw. Vokabeln. Die Hauptansicht besteht aus einer Liste aller erstellten Lernsets. Hier hat der Benutzer Möglichkeit, neue Lernsets anzulegen sowie bestehende Set zu bearbeiten oder zu löschen.

Durch die Auswahl eines bestimmten Lernsets wird eine Liste der dazugehörigen Vokabeln mit ihren jeweiligen Übersetzungen angezeigt. Auch in dieser Ansicht kann der Benutzer neue Vokabeln (entweder einzelne Wörter oder ganze Sätze) hinzufügen sowie vorhandene Einträge bearbeiten oder löschen.

Darüber hinaus kann aus der Ansicht der Vokabelliste direkt mit dem Lernen begonnen werden. Der Benutzer kann dabei zwischen verschiedenen Lernmodi wählen, dem Üben mit Karteikarten oder spielbasierten Übungen.

3.4. Interaktive Lernmodule (Andreleene Prado)

Um Vokabeln effektiv zu lernen, bietet die Anwendung verschiedene interaktive Lernmodule an. Der Benutzer hat dabei die Möglichkeit, entweder mit Karteikarten zu üben oder spielbasierte Lernmodi zu verwenden. Bei den Karteikarten können die Vokabeln in einer ruhigen Lernumgebung wiederholt werden, ohne Zeitdruck oder Punktesystem. Alternativ stehen spielbasierte Übungen zur Verfügung, bei denen die Vokabeln durch verschiedene Aufgabenformate trainiert werden.

3.4.1. Wortbasierte Lernmodi (Andreleene Prado)

Um die Wörter eines Lernsets spielerisch zu üben, bietet die App zwei verschiedene Übungstypen:

- **Paare:** Bei dieser Übung werden mindestens vier Wörter aus dem Lernset verwendet und in zwei Spalten aufgeteilt. Auf der linken Seite werden die deutschen Wörter angezeigt, während sich auf der rechten Seite die entsprechenden Übersetzungen befinden. Die Aufgabe des Benutzers besteht darin, die jeweils passenden Wörtert miteinander zu verbinden.
- **Multiple-Choice Fragen:** Dabei wird dem Benutzer ein Wort angezeigt, zu dem die korrekte Übersetzung ausgewählt werden muss. Neben der richtigen Antwort werden mehrere falsche Antwortmöglichkeiten angezeigt. Der Benutzer muss aus diesen Optionen die passende Übersetzung erkennen und auswählen.

3.4.2. Satzbasierte Lernmodi (Ronja Rüther)

Um neben Wörtern auch Sätze gut trainieren zu können, umfasst die Anwendung drei verschiedene Übungstypen:

- **Lückentext:** Bei dieser Übung fehlt lediglich ein beliebiges Wort im Satz, welches der Nutzer in ein Textfeld eintippen muss. Da diese Übung sich nur auf ein Wort beschränkt und daher eher einfach ist, wird sie nur bei den ersten paar Lernrunden verwendet.
- **Wort-Bausteine:** Der Nutzer setzt den Satz aus vorgegebenen Wort-Fragmenten per Klick zusammen. Dabei sind zusätzliche Wörter, die nicht zum Satz gehören, als Bausteine gegeben, damit es nicht nur ein Puzzle ist. Dadurch wird die Satzstellung geübt, ohne auf die Rechtschreibung achten zu müssen.
- **Freies Schreiben:** Diese Übung ist für Wörter sowie Sätze möglich und ist am anspruchsvollsten, da der gesamte Satz selbstständig in einem Textfeld übersetzt werden muss.

3.4.3. Karteikarten (Andreleene Prado)

Die Karteikarten stellen eine weitere Möglichkeit dar, Vokabeln zu üben, ohne den Druck, Punkte sammeln zu müssen, wie es bei den spielbasierten Übungen der Fall ist. Der Benutzer kann zwischen der vorherigen und der nächsten Vokabel wechseln sowie die Karteikarten neu starten oder zufällig mischen. Außerdem kann ausgewählt werden, ob zuerst die Vokabel oder die entsprechende Übersetzung angezeigt werden soll.

3.5. Chatbot (Andreleene Prado)

Um eine Sprache effektiv zu beherrschen, recht es nicht aus, Vokabeln lediglich auswendig zu lernen. Ebenso wichtig ist es, zu verstehenm wie Wörter in unterschiedlichen Kontexten und Situationen verwendet werden. Der Chatbot-Bereich der Anwendung unterstützt den Benutzer dabei, die gelernten Vokabeln aktiv anzuwenden und in realitätsnahen Szenarien zu üben.

Die Benutzeroberfläche bietet eine Chat-Umgebung, in der der Benutzer zunächst auswählen kann, aus welchem Lernset Vokabeln verwendet werden sollen und welcher Lernmodus genutzt

werden soll. Der Chatbot generiert anschließend passende Aufgaben oder Gesprächssituationen, in denen die ausgewählten Vokabeln angewendet werden müssen.

Der Chatbot biete zwei verschieden Modi an:

- **Satzmodus:** In diesem Modus erstellt der Chatbot einen Satz unter Verwendung von Vokabeln aus dem ausgewählten Lernset. Der Benutzer muss den Satz anschließend in die Zielsprache übersetzen. Nach der Eingabe erhält der Benutzer eine Rückmeldung sowie gegebenenfalls Korrekturen, die auf seiner Antwort basieren. Der Benutzer hat maximal zwei Versuche, um den Satz korrekt zu übersetzen. Ist die Antwort bereits im ersten Versuch korrekt, wird die Vokabel als richtig beantwortet gespeichert. Bei einem Fehler im ersten Versuch erhält der Benutzer zunächst einen Hinweis, der ihm hilft, seine Antwort zu verbessern. Erst nach dem zweiten Versuch wird eine vollständige Bewertung durchgeführt.
- **Konversationsmodus:** In diesem Modus kann der Benutzer ein freies Gespräch mit dem Chatbot führen. Dabei wird der Benutzer dazu aufgefordert, bestimmte Vokabeln aus dem Lernset aktiv in seine Antworten einzubauen, um deren Anwendung zu trainieren. Am Ende des Gesprächs erhält der Benutzer ein zusammenfassendes Feedback, das aufzeigt, welche Aspekte bereits gut beherrscht werden und in welchen Bereichen noch Verbesserungsbedarf besteht.

3.6. Profil (Ronja Rüther)

Das Profil dient als persönliche Leistungsübersicht für den Nutzer. Hier werden alle relevanten Informationen zum Konto sowie die bisherigen Erfolge zusammengefasst:

- **Nutzerdaten:** Neben dem gewählten Profilbild und dem Namen wird auch das aktuelle Level und die Lernserie angezeigt. Wie im Dashboard kann auch hier die Lernsprache über ein Auswahlmenü jederzeit gewechselt werden.
- **Lernstatistik:** Um den Fortschritt des Lernens für den Nutzer sichtbar zu machen, zeigt ein Säulendiagramm die Anzahl der gelernten Vokabeln pro Tag aus den letzten sieben Tagen. Dabei werden nicht nur die richtig beantworteten Vokabeln gezählt, sondern alle insgesamt. Neben den klassischen Lernmodi (wie Paare oder Bausteine) fließen auch das Üben mit Karteikarten oder das Üben im Satzmodus mit dem Chatbot hier rein. Dies soll dem Nutzer helfen, sein Lernverhalten über die Woche hinweg kontrollieren und verfolgen zu können.
- **Abzeichen:** Der Nutzer erhält beim Erreichen bestimmter Ziele Abzeichen, die als zusätzliche Motivation dienen sollen. Wurde ein Abzeichen einmal erhalten, bleibt es für immer bestehen und der Nutzer kann es in diesem Tab immer wieder einsehen.

3.7. Einstellungen (Ronja Rüther)

Mithilfe der Einstellungen kann der Nutzer sein Profil personalisieren. Er kann dort aus einem der 12 vorgefertigten Profilbilder auswählen und Sprachen zu seinem Profil hinzufügen sowie entfernen. Auch wenn aktuell nur die drei Zielsprachen Englisch, Französisch und Spanisch zur Auswahl stehen, werden nicht alle automatisch zum Profil hinzugefügt, um den Nutzer nicht mit irrelevanten Inhalten zu überladen, sodass er im Dashboard oder dem Profil stets zwischen den hinzugefügten Sprachen seine aktuelle Lernsprache auswählen muss.

Zudem bieten die Einstellungen die Möglichkeit, den Nutzer abzumelden und das Profil dadurch vor unbefugtem Zugriff zu schützen. Ebenfalls können die persönlichen Daten aktualisiert werden. Dazu kann der Nutzer über eine Schaltfläche zu einer separaten Ansicht gelangen, in welcher die Aktualisierung des Namens, E-Mail-Adresse und Passworts möglich ist. Hier besteht auch die Möglichkeit, das Benutzerkonto mit allen damit verknüpften Daten und Lernfortschritten unwiderruflich zu löschen.

4. Architektur und Implementierung (Ronja Rüther)

In diesem Kapitel wird die technische Umsetzung von *Lingotrain* erläutert. Dabei wird zunächst die grundlegende Systemarchitektur sowie die Schichtentrennung vorgestellt und anschließend mit der Implementierung der Hauptmodule fortgefahren.

4.1. Systemarchitektur (Ronja Rüther)

Die technische Architektur der Webanwendung basiert auf einer Schichtenarchitektur, um die Benutzeroberfläche von der Geschäftslogik und der Modellhaltung zu trennen. Da die Anwendung mit *Vue.js* umgesetzt wurde, folgt sie einem komponentenbasierten Ansatz mit der Erweiterung um eine zentrale Datenhaltung.

Die Architektur gliedert sich dabei in die folgenden Elemente:

- **View-Layer (Komponenten):** Hierzu zählen die einzelnen Seiten (Views) sowie wiederverwendbare UI-Elemente. Sie sind für die Darstellung von Elementen sowie die Entgegennahme von Nutzerinteraktionen zuständig. Den Einstiegspunkt bildet dabei die `App.vue`, welche das Grundgerüst der Anwendung definiert. Sie steuert den Wechsel der aktuellen Route basierend auf den Metadaten dynamisch zwischen dem *MainLayout* für angemeldete Benutzer und dem *AuthLayout* für Login und Registrierung.
- **Store-Layer:** Dieser Layer dient als zentrale Instanz für die Verwaltung des Anwendungszustands (*State Management*). Er stellt sicher, dass Daten, die komponentenübergreifend benötigt werden, konsistent und reaktiv überall zur Verfügung stehen. Änderungen im Store werden dann direkt an alle betroffenen Komponenten weitergegeben und können so überall gleich angezeigt werden.
- **Repository-Layer:** In den Repositories werden die Datenbankzugriffe auf die Datenbank *Firebase Firestore* gekapselt. Sie enthalten also die einzelnen CRUD-Operationen (Create, Read, Update, Delete), um Daten aus der Datenbank zu lesen und zu aktualisieren. Durch diese Kapselung greifen die Komponenten nie direkt auf die Datenbank zu, sondern nutzen die Methoden der Repositories. Dadurch ist ein einfacher Austausch der Datenbank theoretisch möglich.
- **Model-Layer:** Der Model-Layer umfasst die definierten Datenstrukturen (Interfaces), durch die sichergestellt wird, dass die Objekte über die gesamte Anwendung hinweg dieselbe Struktur besitzen. Dadurch wird für Typsicherheit innerhalb der Anwendung gesorgt und es kann z.B. jedes Konto oder jede Vokabel einheitlich angelegt werden.

Der Datenfluss innerhalb der Anwendung verläuft dabei von der Benutzeroberfläche über den jeweiligen Store zu den Repositories. Die Repositories übernehmen dann die Kommunikation

mit der Datenbank. Nach einer erfolgreichen Datenbankoperation wird der Zustand dann im Store aktualisiert, sodass anschließend auch die reaktiven Komponenten der Benutzeroberfläche automatisch aktualisiert werden.

Neben der Datenhaltung über *Firebase* verfügt die Architektur ebenfalls über eine externe Schnittstelle zu einem separaten Chatbot-Backend. Da dies ausschließlich für die Verarbeitung von Anfragen an ein Sprachmodell (*Large Language Model*) verwendet wird, erfolgt die Kommunikation hierbei über asynchrone *HTTP*-Anfragen direkt aus der Logik der jeweiligen Vue-Komponente heraus.

4.2. Implementierung der Benutzerverwaltung (Ronja Rüther)

Die Benutzerverwaltung wird durch zwei separate Stores realisiert, die eine klare Trennung zwischen der Identität und den Anwendungsdaten sicherstellen.

4.2.1. Authentifizierung (Ronja Rüther)

Der *authStore* ist für die Authentifizierung zuständig und wird mithilfe von *Firebase Auth* umgesetzt. Diese externe Schnittstelle bietet bereits Funktionen zum Anlegen eines Benutzerkontos mit der E-Mail-Adresse und Passwort. Dabei wird das Passwort bereits sicher gespeichert und der Datenbankadministrator erhält keinen Einblick in das verschlüsselte Passwort. Des Weiteren werden Funktionen zum Einloggen eines Nutzers sowie zum Ändern von E-Mail und Passwort bereitgestellt, sodass auf eine bewährte und sicherheitsgeprüfte Implementierung zurückgegriffen werden konnte. Durch die explizite Re-Authentifizierung des Nutzers mittels Passwort können auch diese kritischen Aktionen geschützt werden.

Da *Firebase Auth* ein Konto nur mit E-Mail-Adresse und Passwort anlegt, müssen die zusätzlichen Benutzerdaten und Lernfortschritte zusätzlich in einer Datenbank gespeichert werden. Sollten während der Registrierung Fehler beim Anlegen eines Kontos in der Datenbank *Firebase Firestore* auftreten, wird durch einen Rollback-Mechanismus auch der zugehörige Eintrag in *Firebase Auth* automatisch gelöscht, damit ein konsistenter Datenbestand sichergestellt ist. Durch reaktive Flags wie `aktuellesKonto` wird das aktuell eingeloggte Konto global bereitgestellt.

Die Interaktion mit dem Nutzer erfolgt über Views für Login und Registrierung, sodass diese strikt von der Geschäftslogik getrennt sind. Anstatt die Formulardaten lokal zu verwalten, werden die Eingabefelder direkt an die Datenmodelle des *authStores* gebunden. Über *Lifecycle-Hooks* wie `onMounted` werden während der Registrierung bereits notwendige Daten, wie die verfügbaren Lernsprachen, dynamisch aus dem Repository geladen, um dem Nutzer eine Auswahl anzeigen zu können.

4.2.2. Profil-Logik (Ronja Rüther)

Der *kontoStore* hingegen verwaltet das Konto mit den personalisierten Daten und Lernfortschritten. Er lädt beim Login automatisch die zugehörigen Daten, wie die Sprachobjekte und Abzeichen, aus den entsprechenden Repositories und aktualisiert bei Änderungen diese Daten entsprechend im Repository. Außerdem berechnet er basierend auf den Zeitstempeln, ob die tägliche Lernserie fortgesetzt oder aufgrund einer Pause von mehr als einem Tag zurückgesetzt werden muss.

Das Löschen einer Sprache oder eines Kontos wird hier ebenfalls angestoßen. Da die Lernsets und Vokabeln in unterschiedlichen Collections in *Firestore* gespeichert sind, müssen alle Lernsets und Vokabeln, die mit der Sprache des Kontos bzw. dem gesamten Konto verknüpft sind, rekursiv aus der Datenbank entfernt werden. Dadurch bleibt die Datenintegrität auch beim Löschen von Sprachen oder Konten gewährleistet.

4.2.3. Benutzeroberfläche zur Kontoverwaltung (Ronja Rüther)

Die Interaktion mit den Kontodaten wird über die Komponenten `EinstellungenView.vue` und `KontodatenBearbeiten.vue` realisiert. Um die Benutzerfreundlichkeit und Datensicherheit zu gewährleisten, werden verschiedene *Vue.js*-Konzepte verwendet:

- **Reaktive Datenbindung und Validierung:** In der Bearbeitungsansicht werden die Eingabefelder über ein *reactive*-Objekt verwaltet. Durch einen *Watcher* werden die Änderungen am Formular überwacht, um dem Nutzer durch Methoden wie `validateInputs()` unmittelbar Feedback bei Fehlern zu geben.
- **Computed Properties:** Da in der Datenbank z.B. für Profilbilder und Sprachen nur die IDs gespeichert werden, nutzen die Views *Computed Properties*, um die IDs mit den vollständigen Objekten aus den zugehörigen Stores zu verknüpfen. Dadurch können dem Nutzer immer die korrekten Profilbilder und Flaggen angezeigt werden.
- **Dialog-Komponenten:** Kritische Aktionen wie das Löschen einer Sprache oder eines gesamten Kontos werden über *Alert-Dialoge* gesteuert. Dies verhindert, dass eine Sprache oder ein Konto mit all den zugehörigen Daten versehentlich gelöscht wird.

4.3. Implementierung der Vokabelverwaltung (Andreleene Prado)

Wie bereits im Abschnitt zum Aufbau der Vokabelverwaltung beschrieben, sind Lernsets und Vokabeln in ihrer Struktur ähnlich aufgebaut. Die Darstellung der Lernset- sowie der Vokabelliste erfolgt über entsprechende Views, die mit den jeweiligen Stores verwaltet werden. Die Views greifen dabei direkt auf die im Store gespeicherten Daten zu.

Durch die Verwendung des reaktiven Frameworks Vue.js werden Änderungen automatisch in Benutzerinterface aktualisiert. Sobald sich Daten im Store ändern, beispielweise nach einer Änderung in der Datenbank, wird die Darstellung der Liste automatisch neu gerendert. Dies geschieht nach dem Aufruf der Methoden `findAllLernset()` bzw. `findVokabelnByLernsetId()`. Dadurch ist keine manuelle Aktualisierung der Oberfläche erforderlich.

Da das Laden der Daten aus der Datenbank einige Sekunden dauern kann, wird während des Ladevorgangs ein sogenannter *Skeleton Loader* angezeigt. Dieser wird über ein Flag `loading` im Store gesteuert und signalisiert dem Benutzer, dass die Inhalte gerade geladen werden.

Sowohl für die Lernsetliste als auch für die Vokabelliste stehen Funktionen zum Anlegen, Bearbeiten und Löschen zur Verfügung. Für das Anlegen und Bearbeiten wird derselbe Dialog verwendet, der je nach Zustand unterschiedliche Inhalte anzeigt. Über das Flag `showBearbeiten` wird gesteuert, ob sich der Dialog im Bearbeitungsmodus oder im Modus zum Anlegen eines neuen Eintrags befindet. Beim Bearbeiten werden die bestehenden Daten automatisch in das Formular geladen, während beim Anlegen eines neuen Eintrags ein leeres Formular angezeigt wird.

Die Formularfelder sind mithilfe des reaktiven Vue-Mechanismus implementiert. Die Eingabedaten werden in einem reaktiven Objekt gespeichert, sodass Änderungen direkt mit den entsprechenden UI-Elementen synchronisiert werden. Beim Speichern werden die Daten aus dem Formular ausgelesen und über Funktionen des Stores an die Datenbank übermittelt.

Zusätzlich werden Benutzeraktionen wie das erfolgreiche Speichern oder Löschen eines Lernsets über Benachrichtigungen (*Toast*-Meldungen) bestätigt. Vor dem endgültigen Löschen eines Lernsets wird außerdem ein Bestätigungsdialog angezeigt, um versehentliche Löschvorgänge zu vermeiden.

Damit die Navigation zur richtigen Vokabelliste beim Auswählen eines Lernsets möglich ist, wird die Lernset-ID als Parameter an die entsprechende Route übergeben. Der Name des Lernsets wird zusätzlich mithilfe der Funktion `slugify` in eine URL-freundliche Form umgewandelt, sodass er als lesbarer Bestandteil der URL angezeigt werden kann.

4.4. Implementierung interaktiver Lernmodule (Andreleene Prado)

Die Anwendung bietet verschiedene Lernmodi, die das Üben von Vokabeln abwechslungsreicher und motivierender gestalten sollen. In diesem Abschnitt wird die technische Umsetzung dieser interaktiven Lernmodule beschrieben.

4.4.1. Dynamische Komponentenstruktur der Lernmodule (Andreleene Prado)

Beim Starten des Lernens wird zunächst abgefragt, in welchem Modus der Benutzer die Vokabeln üben möchte. Der ausgewählte Modus wird anschließend zusammen mit wichtigen Informationen, wie der Lernset-ID und einem URL-freundlichen Namen des Lernsets, an die entsprechende Route übergeben.

In der Komponente `ContainerLernen.vue` werden diese Parameter aus der Route ausgelesen und geprüft. Abhängig vom übergebenen Modus wird anschließend dynamisch die passende Komponente geladen. Dabei wird entweder die Komponente `KarteiKarten.vue` für das Lernen mit Karteikarten oder die Komponente `ContainerSpiel.vue` für die spielbasierten Übungen angezeigt. Die Auswahl der Komponente erfolgt über eine berechnete Eigenschaft (*computed property*), die basierend auf dem Modus die entsprechende Kind-Komponente zurückgibt.

Unabhängig davon, welcher Lernmodus ausgewählt wird, wird derselbe übergeordnete Card-Container verwendet. Dadurch wird ein einheitliches Layout für alle Lernmodule gewährleistet und der Aufbau der Benutzeroberfläche bleibt konsistent.

Am oberen Rand des Containers wird eine Progressbar angezeigt, die den aktuellen Lernfortschritt visualisiert. Die Berechnung des Fortschritts erfolgt jedoch innerhalb der jeweiligen Kind-Komponente, da sich die Logik je nach Lernmodus unterscheidet. Der berechnete Fortschrittswert wird anschließend über eine Zwei-Wege-Bindung (`v-model`) an die übergeordnete Komponente zurückgegeben und dort in der Progressbar dargestellt.

Durch diese Architektur wird eine klare Trennung zwischen der allgemeinen Containerstruktur und der spezifischen Logik der einzelnen Lernmodule erreicht.

4.4.2. Implementierung der Karteikarten (Andreleene Prado)

Beim Starten des Karteikartenmodus werden zunächst alle Vokabeln des ausgewählten Lernsets geladen. Dies erfolgt über den *vokabelnStore*, der die Daten aus der Datenquelle abruft und in einer Liste bereitstellt.

Für die Darstellung der Karteikarten wird eine Card-Komponente verwendet. Diese bildet das klassische Layout einer Lernkarte mit Vorder- und Rückseite ab. Durch einen Klick auf die Karte wird diese mittels einer einfachen Rotationsanimation umgedreht. Die Animation wird mit Hilfe von Tailwind CSS umgesetzt, indem eine Transformation entlang der Y-Achse verwendet wird.

Zur Navigation zwischen den einzelnen Karten stehen mehrere Steuerungsbuttons zur Verfügung. Diese ermöglichen das Wechseln zur vorherigen oder nächsten Karte, das Neustarten der Lernrunde sowie das zufällige Mischen der Karten. Zusätzlich kann der Benutzer die Anzeigerichtung ändern, sodass entweder die Vokabel oder die Übersetzung zuerst angezeigt wird.

Zur Verbesserung der Benutzerfreundlichkeit werden die Buttons mit Tooltips versehen. Hierfür werden Komponenten aus der Bibliothek *shadcn-vue* verwendet, insbesondere der *TooltipProvider*. Die Tooltips erklären dem Benutzer die Funktion der jeweiligen Steuerungelemente.

Während der Nutzung wird außerdem der Lernfortschritt berechnet. Dazu wird zunächst die Gesamtanzahl der Vokabeln bestimmt. Bei jeder neuen angezeigten Karte wird der aktuelle Index aktualisert und daraus der Fortschritt in Prozent berechnet. Der berechnete Wert wird anschließend über ein *emit*-Event an die übergeordnete Komponente weitergegeben, welche die Progressbar entsprechend aktualisiert.

4.4.3. Implementierung der Spiellogik (Andreleene Prado)

Die Komponente `ContainerSpiel.vue` enthält die zentrale Spiellogik der Anwendung. Sie steuert den Ablauf der Lernrunde sowie die Auswahl der verschiedenen Spieltypen für Wort- und Satzvokabeln.

Der grundlegende Spielalgorithmus funktioniert nach folgendem Prinzip:

- Zu Beginn einer Lernrunde werden alle noch offenen Vokabeln aus der Datenquelle geladen. Pro Runde werden maximal zehn Vokabeln berücksichtigt.
- Wird eine Vokabel korrekt beantwortet, wird sie als *RICHTIG* markiert und aus der aktuellen Lernrunde entfernt.
- Wird eine Vokabel als falsch beantwortet, wird sie als *FALSCH* markiert und in einer späteren Runde erneut abgefragt.
- Dieser Prozess wird wiederholt, bis alle Vokabeln korrekt beantwortet wurden.

Zunächst werden über den *vokabelKontoStore* alle relevanten Vokabeln für eine Lernrunde geladen. Das zugrunde liegende Repository ruft dabei alle *VokabelKonto*-Einträge ab, deren Status *NICHT_GELERNT* oder *FALSCH* ist und die sowohl zum aktuell angemeldeten Benutzer als auch zum ausgewählten Lernset gehören.

Falls keine offenen Vokabeln gefunden werden, werden alle zugehörigen Einträge automatisch wieder auf den Status *NICHT_GELERNT* gesetzt, sodass eine neue Lernrunde gestartet werden kann.

Auswahl des Spieltyps Für jede Vokabel wird dynamisch ein geeigneter Spielmodus ausgewählt. Die Methode `chooseSpiel()` übernimmt diese Auswahl.

Die Entscheidung basiert auf mehreren Kriterien:

- Handelt es sich bei der aktuellen Vokabel um ein einzelnes Wort, werden ausschließlich wortbasierte Spiele in den Auswahlpool aufgenommen, beispielweise *Paare*, *Multiple-Choice-Fragen* pder *Schreiben*.
- Handelt es sich hingegen um einen vollständigen Satz, werden nur satzbasierte Spiele berücksichtigt, beispielweise *Schreiben*, *Lückentext* oder *Bausteine*.
- Zusätzlich wird über die Methode `getAnzahlGelernt()` ermittelt, wie oft eine Vokabel bereits erfolgreich gelernt wurde. Abhängig von diesem Wert werden einfachere Spieltypen, wie beispielsweise Multiple-Choice-Fragen, aus dem Auswahlpool entfernt, damit der Benutzer stärker zum aktiven Erinnern der Vokabel aufgefordert wird.
- Außerdem wird geprüft, ob genügend Vokabeln für bestimmte Spieltypen vorhanden sind. Wenn beispielweise weniger als drei Wortvokabeln verfügbar sind, wird das Spiel *Paare* aus dem Pool entfernt, da es mit zu wenigen Vokabeln zu einfach wäre.
- Bei Multiple-Choice-Fragen wird ebenfalls geprüft, ob geügend Distraktoren (falsche Antwortmöglichkeiten) vorhanden sind.
- Bei Satzvokabeln wird zusätzlich mit Hilfe der Methode `isSentenceLongEnough()` geprüft, ob der Satz ausreichend lang ist. Kurze Sätze eignen sich nicht für bestimmte Spieltypen wie Lückentexte oder Bausteine. In diesem Fall wird automatisch der Spielmodus *Schreiben* verwendet.
- Falls durch diese Filterung kein Spieltyp mehr verfügbar ist, wird als Fallback ebenfalls der Schreibmodus ausgewählt.
- Anschließend werden ein Spiel zufällig aus der Auswahlpool ausgewählt.

Der ausgewählte Spielmodus wird anschließend dynamisch als Vue-Komponente geladen und im Container angezeigt. Hierfür werden *lazy-loaded* Komponenten verwendet, sodass die jeweiligen Spielmodule erst bei Bedarf geladen werden.

Überprüfung der Antworten Nachdem der Benutzer eine Antwort abgegeben hat, wird diese überprüft und über die Methode `frageBeantwortet()` im *vokabelKontoStore* verarbeitet. Dabei wird der Status der entsprechenden Vokabel in der Datenbank aktualisiert.

Zusätzlich werden Punkte vergeben und gegebenenfalls eine Antwortserie *Streak* erhöht. Bei falschen Antworten wird die Serie zurückgesetzt. Das Ergebnis wird anschließend im Benutzerinterface als Feedback angezeigt.

Nachdem das Feedback angezeigt wurdem kann der Benutzer über den *Next*-Button zur nächste Frage wechseln. Dabei wird automatisch die nächste Vokabel aus der Liste geladen und erneut ein geeigneter Spielmodus ausgewählt.

Berechnung der Progressbar Ähnlich wie in Karteikarten wird auch im Spielmodus der Lernfortschritt in Prozent berechnet und über ein *emit*-Event an die übergeordnete Komponente weitergegeben, welche anschließend die Progressbar aktualisiert.

Im Spielmodus gestaltet sich die Berechnung jedoch etwas komplexer. Insbesondere beim Spieltyp *Paare* werden mehrere Vokabeln gleichzeitig beantwortet. Dabei werden die verwendeten Vokabeln aus der Hauptliste entfernt, wodurch sich die Gesamtanzahl der verbleibenden Vokabeln verändert. Eine Fortschrittsberechnung ausschließlich anhand der aktuell vorhandenen Vokabeln wäre daher ungenau.

Aus diesem Grund werden im *vokabelKontoStore* zusätzliche Zustandsvariablen gespeichert. Zum einen wird die Anzahl der Vokabeln zu Beginn der Lernrunde in der Variable `startAnzahl` festgehalten. Zum anderen wird in der Variable `beantwortet` gezählt, wie viele Vokabeln innerhalb der aktuellen Runde bereits beantwortet wurden.

Der Fortschritt wird anschließend über eine *computed*-Property berechnet, die automatisch auf Änderungen im Store reagiert.

Beenden der Lernrunde Sobald alle Vokabeln der aktuellen Runde bearbeitet wurden, wird die Lernrunde beendet. Dem Benutzer wird anschließend eine Übersicht mit den in dieser Runde erreichten Punkten angezeigt.

Der Benutzer hat danach zwei Möglichkeiten:

- eine neue Lernrunde zu starten oder
- zur Übersicht der Vokabelliste zurückzukehren.

Beim Neustart einer Runde wird die Methode `nextRunde()` aufgerufen. Diese setzt alle relevanten Zustände zurück, lädt die Vokabeln erneut und startet automatisch die erste Frage der neuen Runde.

4.4.4. Wortbasierte Lernmodi (Andreleene Prado)

Für wortbasierte Vokabeln stehen **Multiple-Choice-Fragen** sowie das Zuordnungsspiel **Paare** zur Verfügung.

Im einfachsten Lernmodus werden **Multiple-Choice-Fragen** verwendet. Dabei wird für eine gegebene Vokabel eine korrekte Übersetzung sowie mehrere falsche Antwortmöglichkeiten (sogenannte Distraktoren) angezeigt. Die Distraktoren werden zufällig aus den Vokabeln der aktuellen Lernrunde ausgewählt. Anschließend werden die Antwortmöglichkeiten gemischt und gemeinsam mit der korrekten Lösung dargestellt. Der Benutzer muss anschließend die richtige Übersetzung auswählen.

Die Auswahl der Distraktoren wird durch die Methode `getDistractors()` im *vokabelKontoStore* umgesetzt. Dabei wird zunächst ein Pool möglicher Distraktoren erstellt, der alle Wortvokabeln der aktuellen Runde enthält, jedoch die korrekte Lösung ausschließt. Dieser Pool wird anschließend zufällig gemischt und eine bestimmte Anzahl von Distraktoren wird daraus entnommen.

Der Spielmodus **Paare** stellt eine interaktive Übung dar, bei der Vokabeln ihren passenden Übersetzungen zugeordnet werden müssen. Für dieses Spiel werden mindestens drei und maximal vier Vokabeln aus der aktuellen Lernrunde ausgewählt. Die Auswahl erfolgt über die Methode `getPaare()` aus dem *vokabelKontoStore*. Dabei wird die aktuelle Vokabel stets berücksichtigt, während weitere geeignete Wortvokabeln aus der verbleibenden Liste ergänzt werden.

In der Komponente `PaareSpiel.vue` werden anschließend zwei Listen erzeugt:

- eine linke Liste mit den Vokabeln in der Zielsprache und
- eine rechte Liste mit den zugehörigen Übersetzungen.

Während die linke Liste in ihrer ursprünglichen Reihenfolge angezeigt wird, wird die rechte Liste zufällig gemischt, sodass die Zuordnung nicht unmittelbar erkennbar ist. Der Benutzer muss anschließend die passenden Paare durch eine Drag-and-Drop-Interaktion verbinden.

Um die vom Benutzer erstellten Zuordnungen zu verwalten, werden zwei Mapping-Strukturen verwendet:

- `matches`: speichert die Zuordnung von linker Vokabel zu rechter Übersetzung
- `rightAssigned`: speichert die inverse Zuordnung, um doppelte Zuordnungen zu verhindern

Beim Ziehen einer Vokabel aus der linken Liste wird zunächst deren ID gespeichert. Wird sie anschließend auf eine Übersetzung der rechten Liste abgelegt, wird die entsprechende Zuordnung in beiden Mapping-Strukturen hinterlegt. Dadurch kann jederzeit nachvollzogen werden, welche Paare der Benutzer erstellt hat.

Der Benutzer kann eine Zuordnung jederzeit wieder entfernen, indem er auf die entsprechende Übersetzung klickt. In diesem Fall werden die zugehörigen Einträge aus beiden Mapping-Strukturen gelöscht.

Nachdem alle Paare erstellt wurden, kann der Benutzer seine Antworten überprüfen lassen. Die Methode `pruefen()` vergleicht dabei jede Zuordnung mit der korrekten Lösung.

Für jede linke Vokabel wird geprüft, ob die zugeordnete Übersetzung die gleiche ID besitzt. Das Ergebnis wird in der Struktur `resultMap` gespeichert, die angibt, ob eine Zuordnung korrekt oder falsch ist. Auf dieser Grundlage wird anschließend visuelles Feedback im Benutzerinterface angezeigt, beispielsweise durch farbliche Hervorhebung der richtigen bzw. falschen Zuordnungen. Zusätzlich wird der Lernstatus der jeweiligen Vokabel im *vokabelKontoStore* aktualisiert.

4.4.5. Satzbasierte Lernmodi (Ronja Rüther)

Im Gegensatz zu den wortbasierten Lernmodi müssen bei den satzbasierten Übungen nicht nur einzelne Begriffe, sondern auch die korrekte Satzstellung geprüft werden. Vor der Auswahl eines spielbasierten Modus wird geprüft, ob der Satz ausreichend lang ist. Kürzere Sätze mit weniger als vier Wörtern werden direkt im Modus **Freies Schreiben** geübt, um eine sinnvolle Herausforderung an den Nutzer sicherzustellen.

Im einfachsten Modus **Lückentext** wird ein zufälliges Wort im Satz ausgewählt und durch einen Platzhalter (`_____`) ersetzt. Mithilfe von *Regular Expressions* (Regex) wird sichergestellt, dass das gewählte Wort im Originalstring identifiziert wird. Um nicht immer z.B. das erste Vorkommen eines Wortes im Satz zu ersetzen, werden mittels eines Algorithmus alle Vorkommen des Wortes ermittelt und ein zufälliges davon ersetzt. Dadurch soll die Aufgabe weniger vorhersehbar sein.

Für den **Bausteine**-Modus werden aus dem Originalsatz zunächst die Satzzeichen über eine *Normalisierungsfunktion* entfernt. Anschließend wird der Satz in seine einzelnen Wörter zerlegt. Zur Erhöhung der Schwierigkeitsgrad wird ein *Distraktor-Algorithmus* verwendet, sodass bis zu vier zufällige Wörter ausgewählt werden und dem Nutzer als Bausteine angezeigt werden. Diese zusätzlichen *Distraktor-Vokabeln* stammen aus den übrigen Vokabeln des Lernsets. Dabei wird darauf geachtet, dass es sich um unterschiedliche Wörter handelt und nicht alle Bausteine das

gleiche Wort enthalten. Zuletzt sorgt ein Shuffle-Algorithmus für die zufällige Anordnung der Bausteine, sodass diese möglichst nicht in der Reihenfolge des Satzes vorkommen.

Der letzte satz- sowie wortbasierte Modus ist das **freie Schreiben**. Hierbei ist eine robuste Validierung der Nutzereingabe essenziell, sodass die Eingabe vor dem Vergleich mit dem Originalsatz zunächst neutralisiert wird, um verschiedene Störfaktoren zu entfernen. Dazu zählen mehrfache Leerzeichen oder Zeilenumbrüche. Dadurch wird sichergestellt, dass die Antwort ausschließlich anhand der korrekten Verwendung der Vokabeln bewertet wird und der Nutzer nicht durch die Angabe eines Fehlers nur aufgrund von mehrfachen Leerzeichen verwirrt ist.

Die Steuerung dieser Module erfolgt ebenso wie in den wortbasierten Modulen über das `defineExpose`-Pattern. Dadurch können die Komponenten ihre eigene Logik zum Prüfen der Antwort kapseln und das Ergebnis der übergeordneten Spiel-Steuerung (`ContainerSpiel.vue`) sowie die korrekte Lösung oder Übersetzung für das visuelle Feedback zur Verfügung stellen.

4.5. Implementierung des Chatbots (Andreleene Prado)

Beim Aufrufen des Chatbot-Bereichs wird zunächst eine Auswahl der verfügbaren Lernsets angezeigt, aus denen der Benutzer ein Lernset zum Üben auswählen kann. Zur besseren Übersicht steht dabei eine Suchfunktion zur Verfügung, mit der Lernsets anhand ihres Namens gefiltert werden können. Nachdem ein Lernset ausgewählt wurde, kann der Benutzer zwischen zwei Trainingsmodi wählen: dem **Satzmodus** und dem **Konversationsmodus**.

Satzmodus Im Satzmodus werden fälligen Vokabeln aus dem ausgewählten Lernset über den *vokabelKontoStore* geladen. Für jede Vokabel wird anschließend mit der Methode `loadSentence()` eine passende Aufgabe erzeugt.

Zunächst wird geprüft, ob die aktuelle Vokabel bereits einen vollständigen Satz darstellt oder lediglich ein einzelnes Wort ist. Handelt es sich um einen vollständigen Satz, wird dieser direkt als Übersetzungsaufgabe angezeigt. Ist die Vokabel hingegen ein einzelnes Wort, wird eine Anfrage an das Backend gesendet, um mithilfe eines Sprachmodells einen passenden Beispielsatz zu generieren, der dieses Wort enthält. Der generierte Satz wird anschließend dem Benutzer zur Übersetzung präsentiert.

Die Bewertung erfolgt im Backend, das verschiedene Informationen an das Frontend zurückgibt, darunter:

- ein Feedback zur Antwort,
- eine technische Begründung der Bewertung,
- einen möglichen Verbesserungsvorschlag,
- sowie eine Bewertungskategorie (z. B. *correct*, *almost_correct* oder *wrong*).

Diese Informationen werden anschließend in der Benutzeroberfläche dargestellt, sodass der Benutzer nachvollziehen kann, warum seine Antwort korrekt oder falsch war.

Konversationsmodus Im Konversationsmodus wird eine freiere Form des Sprachtrainings ermöglicht. Hierbei werden ausschließlich Wortvokabeln aus dem ausgewählten Lernset verwendet, während vollständige Sätze ausgeschlossen werden.

Zu Beginn der Konversation wird mit der Methode `startConversation()` eine zufällige Vokabel ausgewählt und an das Backend übermittelt. Zusammen mit der Zielsprache und der Übersetzung dient diese Vokabel als thematischer Fokus für die generierte Unterhaltung.

Das Backend erzeugt daraufhin eine erste Nachricht des Chatbots, die als Ausgangspunkt der Konversation dient. Der Benutzer kann anschließend frei antworten, während das Backend auf Basis des bisherigen Gesprächsverlaufs passende Antworten generiert. Nach Abschluss einer Konversation wird automatisch eine neue Unterhaltung mit einer anderen Vokabel gestartet, sodass mehrere Wörter innerhalb einer Sitzung trainiert werden können.

Verwaltung des Chatverlaufs Alle Nachrichten des Gesprächs werden in einem Array gespeichert, das den gesamten Chatverlauf enthält. Jede Nachricht besitzt dabei eine Rolle (Benutzer oder Chatbot), den eigentlichen Nachrichteninhalt sowie einen Typ, der angibt, ob es sich beispielsweise um eine Aufgabe, eine Antwort oder ein Feedback handelt. Um den Chatverlauf auch nach einem Neuladen der Seite wiederherstellen zu können, wird dieser zusätzlich im *Session Storage* des Browsers gespeichert. Bei jedem Update des Nachrichtenarrays wird der aktuelle Zustand automatisch synchronisiert.

Benutzeroberfläche und Interaktion Die Darstellung des Chats wird über mehrere reaktive Zustandsvariablen gesteuert, die beispielsweise den aktuellen Satz, die Benutzereingabe, das Feedback oder den Ladezustand enthalten. Dadurch passt sich die Benutzeroberfläche automatisch an neue Nachrichten oder Zustandsänderungen an. Zusätzlich wird ein automatisches Scrollen implementiert, sodass der Chatbereich bei neuen Nachrichten immer zum unteren Ende der Unterhaltung springt. Dies wird durch eine Beobachtung der Nachrichtenliste sowie durch eine DOM-Referenz auf den Chatcontainer umgesetzt.

Zurücksetzen der Sitzung Beim Verlassen der Chatbot-Seite wird die aktuelle Sitzung zurückgesetzt. Dabei werden der Chatverlauf gelöscht, alle temporären Zustände der Benutzeroberfläche zurückgesetzt sowie der Eintrag im Session Storage entfernt.

4.6. Visualisierung der Lernstatistik (Ronja Rüther)

Um den Lernfortschritt für den Nutzer sichtbar zu machen, wurde in der `Profil.vue` ein Säulendiagramm mit den Aktivitätsdaten implementiert.

Dafür werden die gelernten Vokabeln pro Tag in der Datenbank als Map (*Key-Value-Paare*) gespeichert, wobei das Datum als Schlüssel und die Anzahl der Vokabeln als Wert dient. Für die Zeitachse werden die Daten der letzten Woche aus der *Computed Property* `letzte7Tage` extrahiert. Fehlende Werte werden mit dem Wert Null initialisiert, damit es zu keinen Fehlern bei der Erstellung des Diagramms kommt und eine korrekte Historie über die letzten 7 Tage auch bei Lernpausen angezeigt werden kann.

Umgesetzt wird das Diagramm mithilfe der Bibliothek *Chart.js*. In der View-Komponente wird dann eine reaktive *BarChart*-Komponente verwendet und über die `chartOptions` wird sichergestellt, dass das Diagramm responsiv auf verschiedene Bildschirmgrößen angepasst wird.

Zusätzlich zur Statistik wird über die Hilfsklasse `LevelCalculator` auch die Gesamtpunktzahl des Nutzers clientseitig in ein Level umgerechnet. Dadurch soll die Datenbank von Zugriffen und redundanten Berechnungen entlastet werden.

4.7. Implementierung des Abzeichensystems (Andreleene Prado)

Zur Motivation der Benutzer wird in der Anwendung ein Abzeichensystem implementiert. Abzeichen werden vergeben, wenn bestimmte Lernziele oder Meilensteine erreicht werden. Geplant ist zunächst die Umsetzung der folgenden Kategorien:

- **Punkte-Abzeichen:** Werden vergeben, sobald eine bestimmte Gesamtpunktzahl erreicht wurde (100, 500, 1000 und 5000 Punkte).
- **Sprachen-Abzeichen:** Wird vergeben, wenn der Benutzer Vokabeln aus mindestens drei verschiedenen Sprachen lernt.
- **Flammen-Abzeichen:** Werden vergeben, wenn eine Lernserie (Streak) von 7 bzw. 30 aufeinanderfolgenden Tagen erreicht wird.

Die Logik zur Vergabe der Abzeichen wird im *abzeichenStore* implementiert. Hierfür wird ein Regelkonzept verwendet, bei dem jede einzelne Abzeichenregel das Interface `AbzeichenRegel` implementiert. Dieses Interface definiert eine einheitliche Struktur für alle Regeln und stellt sicher, dass jede Regel prüfen kann, ob die jeweiligen Bedingungen für ein Abzeichen erfüllt sind. Jede Regel überprüft dabei bestimmte gespeicherte Werte des aktuell eingeloggten Benutzers, beispielsweise die Gesamtpunktzahl, die Anzahl gelernter Sprachen oder die aktuelle Lernserie. Erfüllt ein Benutzer die Bedingungen einer Regel, kann das entsprechende Abzeichen vergeben werden.

Alle definierten Regeln werden in einem Array gespeichert. Dieses Array wird innerhalb der Methode `pruefAbzeichen()` im *abzeichenStore* durchlaufen. Für jede Regel wird überprüft, ob die Bedingungen erfüllt sind und ob das entsprechende Abzeichen dem Benutzer noch nicht zugewiesen wurde. Ist dies der Fall, wird das Abzeichen dem Benutzerkonto hinzugefügt.

Die eigentliche Prüfung der Abzeichen wird durch den *kontoStore* ausgelöst. Hier wird die Methode `checkAbzeichen()` aufgerufen, die wiederum die Abzeichenprüfung im *abzeichenStore* startet.

Diese Prüfung erfolgt automatisch an mehreren Stellen innerhalb der Anwendung:

- beim Aktualisieren der Lernserie (Flamme),
- beim Hinzufügen einer neuen Sprache,
- sowie am Ende jeder abgeschlossenen Lernrunde.

Durch diese automatische Überprüfung wird sichergestellt, dass neue Abzeichen unmittelbar vergeben werden, sobald die entsprechenden Bedingungen erfüllt sind.

5. Aufgetretene Probleme

5.1. E-Mail-Änderung mit Firebase (Ronja Rüther)

Die Änderung der E-Mail-Adresse brachte aufgrund der Sicherheitsvorgaben von *Firebase Auth* und der gewählten Datenbankstruktur mehrere Probleme mit sich.

Ein Problem bestand darin, dass *Firebase Auth* für sensible Aktionen, wie das Ändern der E-Mail-Adresse, einen erneuten Login verlangt. Wenn die Sitzung des Nutzers bereits längere Zeit aktiv war, schlug die Änderung daher ohne explizite Fehlermeldung fehl. Nach erfolgreicher Ermittlung dieser Problemursache, wurde die Implementierung so abgeändert, dass der Nutzer nun immer zur erneuten Eingabe seines Passworts aufgefordert wird. Durch die Methode `reauthenticateWithCredential(...)` wird sichergestellt, dass der Auth-Token aktuell ist und der Nutzer re-authentifiziert wird.

Zusätzlich erschwert die Sicherheits-Logik von *Firebase Auth* das Feedback an den Nutzer. Da aus Datenschutzgründen bei einer E-Mail-Änderung mit `verifyBeforeUpdateEmail(...)` nicht direkt geprüft werden kann, ob eine E-Mail bereits bei einem anderen Konto existiert, erhält der Nutzer in dem Fall keine E-Mail zur Bestätigung.

Des Weiteren wird die E-Mail-Adresse sowohl in *Firebase Auth* als auch in der Datenbank im *Firestore*-Dokument des Kontos gespeichert. Da die Änderung der E-Mail-Adresse erst nach erfolgreicher Verifizierung der neuen E-Mail in *Firebase Auth* aktualisiert wird, kann die Änderung der E-Mail nicht zuvor schon in der *Firestore*-Datenbank vorgenommen werden. Um dies zu lösen, wird der Nutzer nach einer E-Mail-Änderung zunächst automatisch ausgeloggt. Beim nächsten Login wird dann geprüft, ob die neue E-Mail-Adresse verifiziert wurde, um sie dann entsprechend in der Datenbank zu aktualisieren. Durch dieses Vorgehen wird eine verzögerte Synchronisation sichergestellt, sodass spätestens beim nächsten Login die Datenkonsistenz zwischen dem Authentifizierungssystem und der Datenbank wiederhergestellt ist.

5.2. Globales Laden von Lernsets (Ronja Rüther)

Eine weitere Herausforderung bei der Entwicklung war die Bereitstellung der nutzer- sowie sprachenspezifischen Lernsets. Nach dem Start oder Neuladen der Anwendung, aber auch nach dem Wechsel der aktuellen Lernsprache wurden zunächst nicht die korrekten Lernsets in den verschiedenen Tabs geladen.

Anfangs wurden die Sprachen eines Kontos über den Aufruf von `loadSprachenZuKonto(...)` des *kontoStores* lokal in der Komponente `Home.vue` geladen. Im Dashboard waren daher die Shortcuts zu den Lernsets an die ausgewählte Sprache angepasst. Wurde die Anwendung über eine direkte URL einer Unterseite gestartet oder die Unterseiten aktualisiert, blieb der Datenbestand leer. Zudem wurden nach einem Sprachwechsel teilweise noch die Lernsets der vorherigen Sprache angezeigt, da der lokale State nicht zuverlässig aktualisiert wurde.

Um dieses Problem zu lösen und die Lernsets konsistent über alle Komponenten verfügbar zu stellen, wurde die Lade-Logik zentralisiert. In der `App.vue` wurde ein globaler Watcher implementiert, der die Zustände der aktuellen Konto-ID, der aktuellen Lernsprache und der Liste der hinzugefügten Sprachen überwacht. Durch die Konfiguration `immediate: true` ist sichergestellt, dass der Watcher direkt beim Start der Anwendung reagiert und die Sprachen und

Lernsets nach der Authentifizierung automatisch geladen werden. Dadurch entstehen keine leeren Datenbestände beim direkten Aufruf von Unterseiten und auch beim Sprachwechsel werden die entsprechenden Lernsets synchronisiert.

5.3. Aktualisierung und Berechnung der Progressbar (Andreleene Prado)

Ein zentrales Problem während der Entwicklung trat bei der korrekten Darstellung der Progressbar auf, insbesondere im Spielmodus der Anwendung. Während die Berechnung des Lernfortschritts im Karteikartenmodus relativ einfach umgesetzt werden konnte, erwies sich dies bei den Spielmodi als deutlich komplexer.

Im Karteikartenmodus wird der Fortschritt berechnet, indem der Index der aktuell angezeigten Vokabel durch die Gesamtanzahl der Vokabeln der aktuellen Lernrunde geteilt wird. Das Ergebnis wird anschließend als Prozentwert berechnet und an die übergeordnete Komponente übergeben, welche die Progressbar darstellt.

Im Spielmodus, insbesondere beim *Paare*-Spiel, gestaltet sich die Berechnung des Fortschritts komplizierter. Hier werden mehrere Vokabeln gleichzeitig in einer Spielrunde verwendet. Zudem müssen die verwendeten Vokabeln nach Abschluss des Spiels aus der Liste der verbleibenden Vokabeln entfernt werden, damit sie innerhalb derselben Lernrunde nicht erneut erscheinen.

Zu Beginn der Implementierung wurde die Progressbar im Spielmodus nach dem gleichen Prinzip wie im Karteikartenmodus berechnet. Nachdem jedoch das Paare-Spiel vollständig integriert wurde, zeigte sich, dass die Fortschrittsanzeige nicht mehr korrekt funktionierte. Der Fortschritt bewegte sich teilweise unregelmäßig oder sprang scheinbar zufällig, da mehrere Vokabeln gleichzeitig verarbeitet und aus der Liste entfernt wurden.

Um dieses Problem zu lösen, wurden zusätzliche Zustandsvariablen eingeführt. Eine Variable speichert die Gesamtanzahl der Vokabeln zu Beginn der Lernrunde, während eine zweite Variable die Anzahl der bereits beantworteten Vokabeln erfasst. Nach jeder abgeschlossenen Aufgabe wird die Anzahl der beantworteten Vokabeln entsprechend erhöht. Der Fortschritt der Progressbar wird anschließend anhand diese Werte in Prozent berechnet. Durch diese Anpassung bleibt die Fortschrittsanzeige stabil und korrekt, unabhängig davon, welcher Lernmodus verwendet wird oder wie viele Vokabeln innerhalb einer Spielrunde verarbeitet werden.

5.4. Anzeige des Level-Up-Overlays (Ronja Rüther)

Ein weiteres Problem ergab sich bei der Anzeige des Level-Up-Overlays nach dem Abschluss einer Lernrunde. Erreicht ein Nutzer durch die in der Runde gesammelten Punkte ein neues Level, soll ein visuelles Overlay den Levelaufstieg anzeigen.

Wird die Lernrunde regulär beendet, kann das Overlay problemlos dargestellt werden. Verlässt der Nutzer die Lernrunde jedoch vorzeitig, z.B. durch eine Navigation zu einer anderen Seite, wird die aktuelle Komponente durch den Routenwechsel zerstört. Die Punkte der Runde werden in diesem Fall zwar weiterhin korrekt im Store berechnet und dem Konto hinzugefügt, aber das Overlay kann nicht mehr angezeigt werden, da die dafür zuständige Komponente nicht mehr aktiv ist.

Als möglicher Lösungsansatz wurde versucht, den Seitenwechsel mit dem *Vue-Router-Navigation-Guards* `onBeforeRouteLeave` abzufangen und das Overlay vor dem Wechsel der Navigationsroute

anzuzeigen. Dabei sollte der Wechsel der Route verzögert werden. Dieser Ansatz konnte jedoch nicht erfolgreich umgesetzt werden, da der ursprünglich ausgelöste Seitenwechsel nach dem Schließen des Level-Up-Overlays nicht mehr ausgeführt wurde. Der Nutzer blieb stattdessen auf der aktuellen Seite.

Da die erreichten Punkte trotzdem korrekt gespeichert werden und daher auch das Level entsprechend aktualisiert wird, wurde auf eine weitergehende Lösung verzichtet. Da das Problem zudem nur auftritt, wenn die Lernrunde vorzeitig verlassen wird, wird dieses Verhalten als akzeptabel eingestuft.

6. Ergebnisse und Fazit (Andreleene Prado)

Zusammenfassend kann festgestellt werden, dass die Implementierung des Vokabeltrainers erfolgreich umgesetzt wurde. Die entwickelte Webanwendung ermöglicht es Benutzern, eigene Vokabeln in Lernsets zu organisieren und diese jederzeit flexibel zu üben. Hierfür stehen verschiedene Lernmethoden zur Verfügung, darunter klassische Karteikarten sowie mehrere spielerische Lernmodi, die das Wiederholen der Vokabeln abwechslungsreicher gestalten.

Ein besonderes Merkmal der Anwendung ist die Integration eines Chatbots, der es den Benutzern ermöglicht, die gelernten Vokabeln aktiv in Sätzen oder in kurzen Konversationen anzuwenden. Dadurch wird nicht nur das reine Auswendiglernen unterstützt, sondern auch die praktische Anwendung der Sprache gefördert.

Zusätzlich wurden verschiedene Gamification-Elemente integriert, wie beispielsweise ein Punktesystem, Abzeichen und Lernserien. Diese Elemente dienen dazu, die Motivation der Benutzer zu steigern und langfristiges Lernen zu fördern. Durch das Sammeln von Punkten und das Freischalten von Abzeichen werden Lernfortschritte sichtbar gemacht und zusätzliche Anreize zum regelmäßigen Üben geschaffen.

Die technische Umsetzung mit dem Framework Vue.js erwies sich als besonders vorteilhaft. Durch das komponentenbasierte Architekturmodell konnten die einzelnen Funktionen der Anwendung klar strukturiert und modular entwickelt werden. Die Verwendung von reaktiven Zuständen und zentralen Stores erleichterte außerdem die Verwaltung von Benutzerdaten, Lernfortschritten und Spielzuständen innerhalb der Anwendung.

Die entwickelte Anwendung bietet eine solide Grundlage, die in Zukunft weiter ausgebaut werden kann, beispielsweise durch zusätzliche Lernmodi, weitere Sprachen oder eine erweiterte Personalisierung des Lernprozesses.

7. Anhang (Ronja Rüther)

7.1. Verwendete grafische Assets

Einige grafische Elemente der Anwendung wurden mithilfe des KI-Tools ChatGPT generiert. Dazu zählen das App-Logo, Profilbilder, Abzeichen sowie Streak-Icons.

Weitere verwendete Grafiken stammen aus öffentlich zugänglichen Quellen:

- Flagge von England - Wikipedia (https://de.wikipedia.org/wiki/Datei:Flag_of_the_United_Kingdom.png)
- Flagge von Frankreich - Wikipedia (https://de.wikipedia.org/wiki/Datei:Flag_of_France.png)
- Flagge von Spanien - Wikipedia (https://de.wikipedia.org/wiki/Datei:Flag_of_Spain.svg)
- Icon für den Tab „Meine Vokabeln“ - Google Material Icons (https://fonts.google.com/icons?selected=Material+Icons:library_books:&icon.query=learning&icon.size=24&icon.color=%23FFFFFF&icon.platform=web)

Literatur

[1] C. Blanco. „Duolingo für Anfänger: Wie lernt man eine Sprache auf Duolingo?“ (2025), Adresse: https://blog.duolingo.com/de/duolingo-fur-anfanger-wie-man-eine-sprache-auf-duolingo-lernt/ (besucht am 06. 03. 2026).

[2] Vue.js. „Introduction.“ (), Adresse: https://vuejs.org/guide/introduction.html (besucht am 09. 03. 2026).

[3] Vue.js. „Routing.“ (), Adresse: https://vuejs.org/guide/scaling-up/routing (besucht am 09. 03. 2026).

[4] W. contributors. „Tailwind CSS.“ (2026), Adresse: https://en.wikipedia.org/wiki/Tailwind_CSS (besucht am 09. 03. 2026).

[5] Pinia. „Introduction | What is Pinia?“ (), Adresse: https://pinia.vuejs.org/introduction.html (besucht am 09. 03. 2026).

[6] geeksforgeeks. „FastAPI - Introduction.“ (2025), Adresse: https://www.geeksforgeeks.org/python/fastapi-introduction/ (besucht am 09. 03. 2026).